Phase 4 | Interview Question Bank

Practice project, architecture, database, authentication, API, and reliability questions.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

Project framing

Q: What is Ready Scout?

A: A disaster-preparedness learning platform built around a React/Vite frontend, Express API and MongoDB Atlas. It includes learning content, an earthquake simulation, alert views, nearby resource lookup and a Flask/Gemini chatbot. It is an educational prototype, not an emergency dispatch system.

Q: Who is the primary user?

A: Learners, especially students exploring preparedness content and interactive practice. The repository also has a teacher-facing route, but the product claim should reflect the feature scope actually demonstrated.

Q: What is the main user journey?

A: Open the web app, browse learning resources, sign in to save account state, use alerts and nearby services as references, complete earthquake learning activities, and ask the chatbot follow-up questions.

Architecture and frontend

Q: Why React with Vite?

A: React provides component composition and Vite provides a fast dev server and static production build. This project uses React Router for navigation and has a separate Express API, so a client-rendered SPA is a straightforward fit.

Q: What must production hosting support?

A: It must serve the Vite build, rewrite frontend routes to index.html, and route /api and /uploads to Express. Environment URLs must match the deployed service origins.

Q: How does the app manage login state?

A: AuthContext loads stored user data from localStorage, updates state after login/register, and sends a token on protected calls. This is convenient but less resistant to XSS than HttpOnly cookie storage.

Backend, data and security

Q: How are passwords stored?

A: The User model hashes passwords using bcryptjs in a pre-save hook. Login compares the submitted password against the stored hash.

Q: How is a protected request authorized?

A: The client sends Authorization: Bearer token. Middleware verifies the JWT with the server secret, loads the user and attaches it to the request before calling the protected route.

Q: What does MongoDB Atlas store?

A: The Mongoose User document contains profile identity, email, username, password hash, role, picture path, optional Google ID and timestamps.

Q: Why use an ODM?

A: Mongoose provides schema definitions, model methods, validation hooks and a consistent access layer over MongoDB. It does not replace database indexes or authorization checks.

Q: How would you improve token handling?

A: Avoid putting the access token in the OAuth redirect URL, use secure HttpOnly cookies or a short-lived one-time code exchange, and implement a properly signed, revocable refresh-token flow.

Providers and product reliability

Q: What is the difference between Leaflet and Geoapify?

A: Leaflet renders the map. Geoapify Places searches for nearby points. OpenStreetMap is the underlying place/map data ecosystem; the provider and renderer are different components.

Q: Why can the locator fail?

A: It depends on a backend API key, provider permissions/quota, category coverage, network availability and correct deployment environment configuration.

Q: How is CAP alert detail displayed?

A: Express fetches CAP XML, parses it, selects a language-specific info block, and returns structured fields as JSON for the detail page.

Q: How does the chatbot work?

A: React sends a message to Flask; Flask adds a disaster-education system instruction, calls Gemini generateContent and returns candidate response text.

Q: How would you test provider failure?

A: Use mocked upstream responses for success, timeout, malformed JSON and rate-limit/auth errors. Verify safe user-facing messages, logs without secrets and stale-cache behavior.

Q: What is the most important limitation?

A: External sources can be stale, incomplete or unavailable. The app must not imply guaranteed emergency facility coverage or replace official alerting channels.